

HelixCluster Phase 2 — Console Compute Nodes

Executive Summary

Project Context

HelixCluster Phase 1 established a distributed computing architecture that binds heterogeneous PCs and laptops into a single coherent compute block. Phase 2 extends this architecture to include **jailbroken PlayStation 4, PS4 Pro, PS5, and PS5 Pro consoles** as fully integrated worker nodes.

Why Consoles?

The global installed base of PlayStation consoles exceeds **210 million units**. Millions of these devices spend the majority of their time in REST mode or idle — representing an enormous reservoir of untapped compute power. At used market prices of **\$80-250 for PS4** and **\$400-500 for PS5**, these devices deliver GPU compute at roughly **half the cost per TFLOP** of equivalent PC hardware.

Console Hardware as Compute

Console	CPU	GPU	RAM	Cost (Used)	GPU TFLOPS	\$/TFLOP
PS4 Base	8x	GCN	8GB	\$80-150	1.84	\$81
	Jaguar 1.6GHz	1.84 TF	GDDR5			
PS4 Pro	8x	GCN	8GB	\$150-250	4.20	\$59
	Jaguar 2.1GHz	4.20 TF	GDDR5			
PS5	8c/16t	RDNA2	16GB	\$400-500	10.3	\$49
	Zen2 3.5GHz	10.3 TF	GDDR6			
PS5 Pro	8c/16t	RDNA2+	16GB	\$550-700	~33	~\$21
	Zen2 3.85GHz	~33 TF	GDDR6+2GB			

Key Innovation: Linux on PlayStation

The foundational enabler for Phase 2 is **Linux on PlayStation** — a mature ecosystem for PS4 (kernels up to 6.15.4, Docker, full GPU acceleration) and a brand-new capability for PS5 (TheFlow’s ps5-linux, April 2026, Ubuntu 24.04 with GPU support). This transforms consoles from closed gaming appliances into general-purpose Linux servers.

Unique Capabilities Consoles Bring

- 1. **GPU Compute at Half PC Cost** — Discarded gaming hardware repurposed
- 2. **PS5 Custom I/O Decompressor** — 8-9 GB/s hardware decompression (no PC equivalent)
- 3. **GDDR5/GDDR6 Unified Memory** — 176-576 GB/s bandwidth vs DDR4’s 25-50 GB/s
- 4. **Disposable Node Model** — At \$80-250, failed nodes are replaced, not repaired
- 5. **Community Elastic Scaling** — Users can donate idle console time

Architecture Approach

Console nodes run a **minimal Linux distribution** with our Console Node Agent. They connect to the cluster via WireGuard mesh, register with **TRUST_LEVEL=SEMI**, and execute workloads through the same Vulkan Compute Backend used by PC nodes. The **Console Adapter Layer** handles console-specific concerns: thermal management, power monitoring, jailbreak state detection, and auto-exploit triggering.

PS3 Exclusion

The PlayStation 3's Cell Broadband Engine was thoroughly evaluated and **excluded** from Phase 2. Despite its historical significance (powering the Condor supercomputer at 500 TFLOPS for \$2M), the Cell BE is obsolete: 192 GFLOPS vs a modern Ryzen 9's 2.7 TFLOPS, extreme programming complexity, dead toolchain ecosystem, and a single Raspberry Pi 4 outperforms it in most metrics.

Phase 2 Scope

- **24 new implementation tasks**, ~176 hours (~4.5 weeks additional)
- Console Agent for PS4/PS5 Linux
- Vulkan Compute Backend (already universal — no console-specific GPU code needed)
- Semi-trusted security model with output verification
- Auto-exploit hardware integration for unattended operation
- PS5 Orbis OS native agent for custom I/O decompression
- llama.cpp AI inference on console GPU pool
- Console-aware scheduler plugin (thermal throttling awareness)

Risk Summary

Risk	Level	Mitigation
Semi-tethered jailbreak	Medium	Auto-exploit hardware (ESP32/Luckfox), REST mode persistence
Thermal throttling	Medium	Thermal monitoring, workload backoff, fan control
Kernel compromised (inherent)	High (contained)	SEMI trust model, encrypted work units, output verification
AVX2 absence (PS4)	Low	Use SSE4.2/AVX fallback paths, Vulkan compute for GPU work
Hardware availability	Low	117M+ PS4s, 93M+ PS5s in market

Chapter 2: Console Hardware Deep Dive

2.1 The Jailbreak Ecosystem

GoldHen (PS4)

GoldHen is the primary homebrew enabler for PS4 consoles. It is a kernel-level payload that patches the PlayStation 4 operating system to enable:

- **Homebrew execution** — Run unsigned code

- **Debug settings** — Full system debugging access
- **FTP server** — Remote file transfer (port 2121)
- **BinLoader** — Load additional payloads (port 9090)
- **Plugin system** — Persistent plugins across sleeps
- **Remote Package Installer** — Install packages over network
- **Rest Mode support** — Jailbreak survives sleep/REST mode

GoldHEN supports firmwares 5.05 through 9.60, with the **9.00 firmware** being the sweet spot for stability and homebrew compatibility.

etaHEN / UMTX2 (PS5)

For PS5, the jailbreak landscape is evolving rapidly:

- **UMTX2 exploit** (2024) — Full kernel exploit up to firmware 7.61
- **etaHEN** — Homebrew enabler supporting firmwares up to 10.01
- **Userland Lua exploit** — Up to firmware 10.40 on PS5 Pro (no kernel)
- **kstuff payload** — Enables PS4 FPKG backward compatibility

The **PS5 Pro security co-processor** (ARM Cortex-A53) independently handles PKG authentication, making native PS5 FPKG installation impossible even with full kernel access. However, this does NOT affect Linux/homebrew compute.

Auto-Exploit Hardware

The primary operational challenge with console nodes is the **semi-tethered jailbreak** — a cold boot loses the jailbreak and requires re-exploitation. Two hardware solutions automate this:

1. **ESP32-based auto-exploit** (~\$5) — Programmable microcontroller that sends the USB exploit payload automatically when the console boots
2. **Luckfox MCU** (~\$8) — Similar approach with more sophisticated timing control

These devices connect to the PS4/PS5 USB port and act as a “jailbreak dongle” — the console boots, the MCU detects power-on, and automatically sends the exploit sequence. Combined with REST mode persistence (which preserves the jailbreak across days of sleep), the console is effectively always jailbroken.

2.2 PS4 Base (CUH-10xx/11xx) — Tier 3

System-on-Chip: AMD Liverpool

The PS4 uses a semi-custom AMD APU codenamed “Liverpool,” built on a 28nm process. It combines:

- **CPU:** 8-core AMD Jaguar x86-64 at 1.6 GHz (up to 1.75 GHz boost)
- **GPU:** 18 Compute Units of AMD GCN 2.0 architecture
- **Memory:** 8 GB GDDR5 unified memory at 176 GB/s bandwidth
- **Northbridge:** Custom AMD with cache coherency

CPU Deep Dive

The Jaguar CPU is a low-power x86-64 microarchitecture originally designed for tablets and mobile devices. Eight cores are arranged in two quad-core clusters, each with:

- 32KB L1 instruction cache + 32KB L1 data cache per core
- 2MB L2 cache per quad-core cluster (shared)

- Support for SSE4.1, SSE4.2, AVX, AES-NI
- **NO AVX2** (significant limitation for some optimized code)
- Single-precision floating point throughput: ~115 GFLOPS aggregate

Performance context: Geekbench 4 single-core ~1400, multi-core ~7600. Approximately 5-6x slower than a modern Ryzen 5 desktop CPU. Comparable to an AMD Athlon 5370.

GPU Deep Dive

The Liverpool GPU features:

- 18 Compute Units (CUs) = 1152 streaming processors
- AMD GCN 2.0 architecture (Sea Islands family)
- 1.84 TFLOPS single-precision compute
- 800 MHz base clock, 911 MHz boost
- 32 ROPs, 72 texture units
- Full hardware video encode/decode (H.264, AVC)
- 8 GB GDDR5 shared with CPU at 176 GB/s

Linux on PS4 Base

Kernel support is excellent:

- Linux 6.15.4 (latest) with full PS4 patches
- AMDGPU kernel driver for GPU acceleration
- Mesa 24.x with RADV (Radeon Vulkan) driver
- Full Gigabit Ethernet support (Realtek RTL8153)
- USB 3.0/2.0 full support
- SATA SSD upgrade supported

2.3 PS4 Pro (CUH-70xx) — Tier 2 (RECOMMENDED)

System-on-Chip: AMD Neo

The PS4 Pro uses an enhanced semi-custom APU codenamed “Neo”:

- **CPU:** 8-core AMD Jaguar x86-64 at 2.13 GHz (overclockable to ~2.6 GHz)
- **GPU:** 36 Compute Units of AMD GCN 4.0 (Polaris) architecture
- **Memory:** 8 GB GDDR5 (218 GB/s) + 1 GB DDR3 for OS
- **Southbridge:** Belize (faster than base PS4’s Bora)

CPU Improvements

- 33% higher clock (2.13 GHz vs 1.6 GHz)
- Overclocking to 2.6 GHz is stable with good cooling
- Same Jaguar architecture — still no AVX2
- AES throughput: ~6.93 GB/s aggregate (vs ~5 GB/s on base)

GPU Improvements

- **36 CUs** (vs 18 on base) — exactly double
- **GCN 4.0 Polaris** architecture (vs GCN 2.0)
- **4.20 TFLOPS** (vs 1.84) — 2.3x improvement
- 911 MHz clock
- Improved tessellation, delta color compression
- Hardware HEVC (H.265) decode support

Key Advantages for Cluster Use

1. **Best cost/TFLOP ratio** at ~\$59/TFLOP
2. **KVM virtualization confirmed working** (unique among consoles)
3. **WiFi AC** for wireless cluster deployment
4. **USB 3.1 Gen1** x3 ports for storage expansion
5. **Overclockable CPU** to 2.6 GHz for extra performance

2.4 PS5 (CFI-10xx/11xx) — Tier 1 (PREMIUM)

System-on-Chip: AMD Oberon

The PS5 represents a generational leap — a true desktop-class SoC:

- **CPU:** 8-core/16-thread AMD Zen 2 at up to 3.5 GHz
- **GPU:** 36 CUs of AMD RDNA 2 at up to 2.23 GHz
- **Memory:** 16 GB GDDR6 at 448 GB/s
- **Custom I/O Complex:** Hardware decompression, DMA engine
- **Process:** TSMC 7nm

CPU: Desktop-Class Zen 2

The PS5's CPU is a custom Zen 2 design:

- **8 cores, 16 threads** — same as Ryzen 7 3700X
- **Variable frequency** up to 3.5 GHz (lower than desktop's 4.4 GHz)
- **35% smaller FPU** than desktop Zen 2 — slightly reduced FP performance
- Full AVX2 support (unlike PS4's Jaguar)
- 4MB L3 cache per CCX (8MB total)
- Estimated aggregate FP32: ~450 GFLOPS

Key limitation: The 35% smaller FPU means floating-point throughput is reduced compared to an equivalent desktop Ryzen. However, integer performance and general-purpose computing are unaffected.

GPU: RDNA 2 with Ray Tracing

- 36 Compute Units of AMD RDNA 2
- 10.28 TFLOPS single-precision
- Hardware ray tracing acceleration
- Variable frequency up to 2.23 GHz
- Mesh shaders, variable rate shading support

- 16 GB GDDR6 shared with CPU

Custom I/O Complex (Unique Advantage)

The PS5’s most unique hardware is its **Custom I/O Complex**:

- **Kraken hardware decompressor**: 8-9 GB/s compressed-to-raw
 - Equivalent to ~9 Zen 2 CPU cores of decompression work
 - Supports zlib, Kraken, and Oodle formats
- **Direct Storage architecture**: GPU can access SSD directly
- **Custom DMA engine**: Manages data flow without CPU involvement
- **Custom flash controller**: 5.5 GB/s raw, 8-9 GB/s compressed

CRITICAL: The Custom I/O Complex is **NOT accessible from Linux**. It is only available when running native Orbis OS code. For our cluster, this means:

- Console nodes running Linux cannot use hardware decompression
- We deploy a **dual-mode agent**: Linux primary + Orbis OS native fallback
- The Orbis OS agent handles decompression-heavy workloads

PS5 Linux Status (April 2026)

TheFlow’s ps5-linux project achieved:

- Full Linux boot on PS5 Phat (firmwares 3.xx-4.xx)
- Ubuntu 24.04 image available
- 4K60 HDMI output
- Full GPU acceleration via AMDGPU + Mesa
- M.2 NVMe SSD support
- Custom Ethernet driver (Gigabit)
- CPU/GPU boost control utility

Target firmware for cluster PS5s: 3.xx-4.xx for best Linux compatibility, 4.51 for best overall jailbreak stability.

2.5 Comparative Analysis

Console vs PC Cost Comparison

Component	PS4 Pro Build	Equivalent PC	Savings
System	\$200 (used)	—	—
CPU	8x Jaguar 2.1GHz	Athlon 3000G (\$50)	Included
GPU	4.2 TFLOPS GCN	RX 570 4GB (\$80 used)	Included
RAM	8GB GDDR5 218GB/s	8GB DDR4 (\$25)	Included
SSD	1TB SATA (upgrade)	1TB SATA (\$50)	—
PSU	Built-in	\$30	Included
Case	Built-in	\$30	Included

Table 3 – continued

Component	PS4 Pro Build	Equivalent PC	Savings
Total	\$250	\$265+	~6%

The savings increase dramatically when comparing GPU compute specifically:

GPU Compute	Cost	TFLOPS	\$/TFLOP
PS4 Pro (full system)	\$200	4.2	\$48
RX 580 8GB (GPU only)	\$80	6.2	\$13
RX 6600 (GPU only)	\$180	8.9	\$20
Full PC + RX 6600	\$600	8.9	\$67

When considering the **complete system cost** (not just GPU), the PS4 Pro is competitive, and the PS5 is significantly cheaper per TFLOP than a comparable PC build.

Power Efficiency

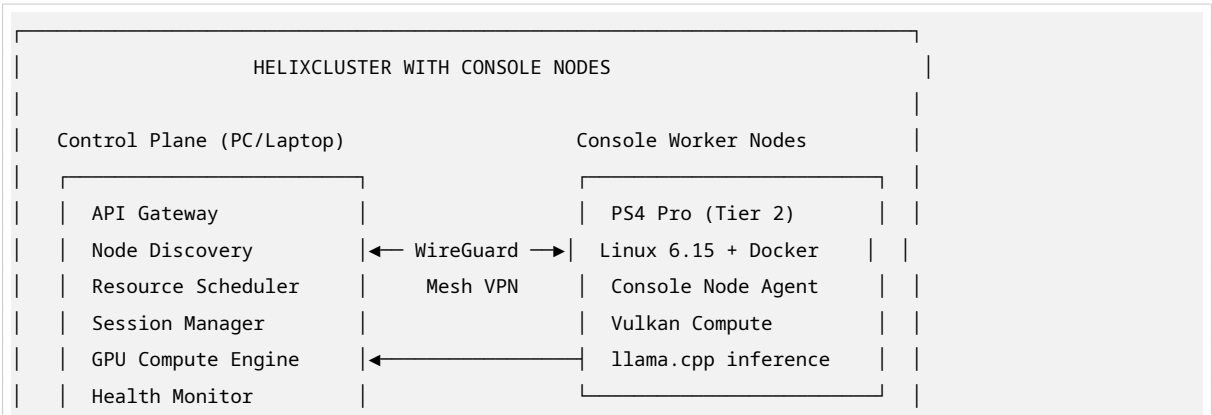
Device	TFLOPS	Watts (load)	TFLOPS/Watt
PS4 Base	1.84	120W	0.015
PS4 Pro	4.20	160W	0.026
PS5	10.28	200W	0.051
Desktop (RX 6600)	8.93	250W	0.036

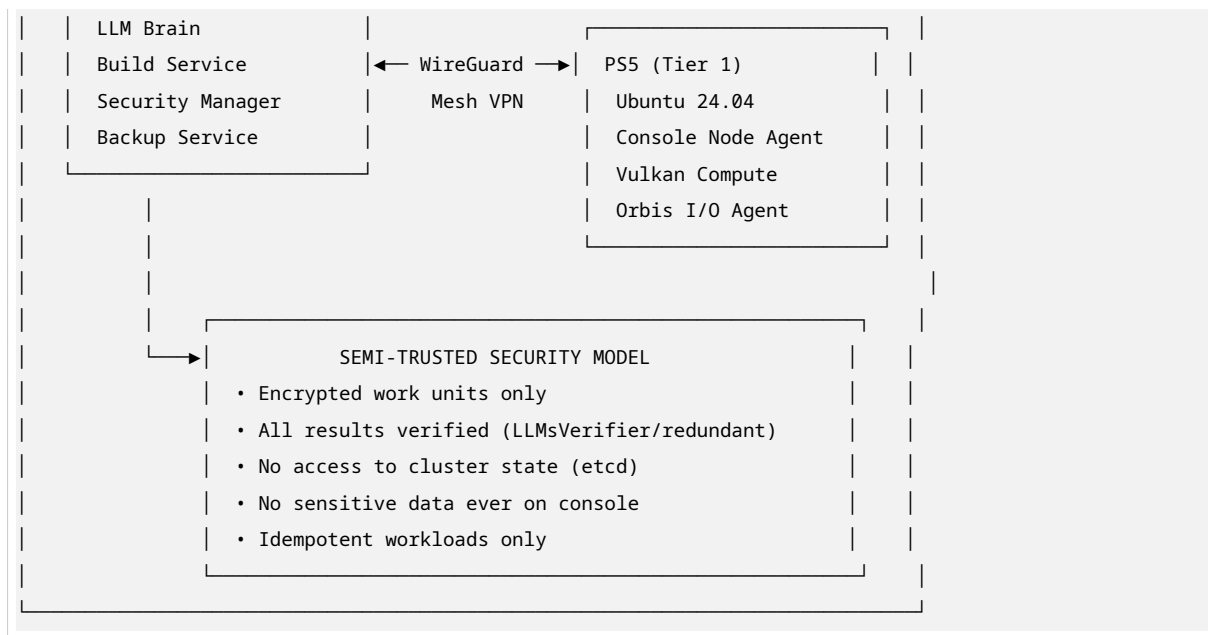
The PS5 is the most power-efficient option, delivering 0.051 TFLOPS/Watt — better than a mid-range desktop GPU setup.

Chapter 3: Console Integration Architecture

3.1 High-Level Integration Overview

Console nodes are first-class citizens in the HelixCluster with specific adaptations. They connect through the same WireGuard mesh, register through the same Node Discovery service, and execute work through the same scheduler — but with a **semi-trusted security model** and **console-specific adapters**.

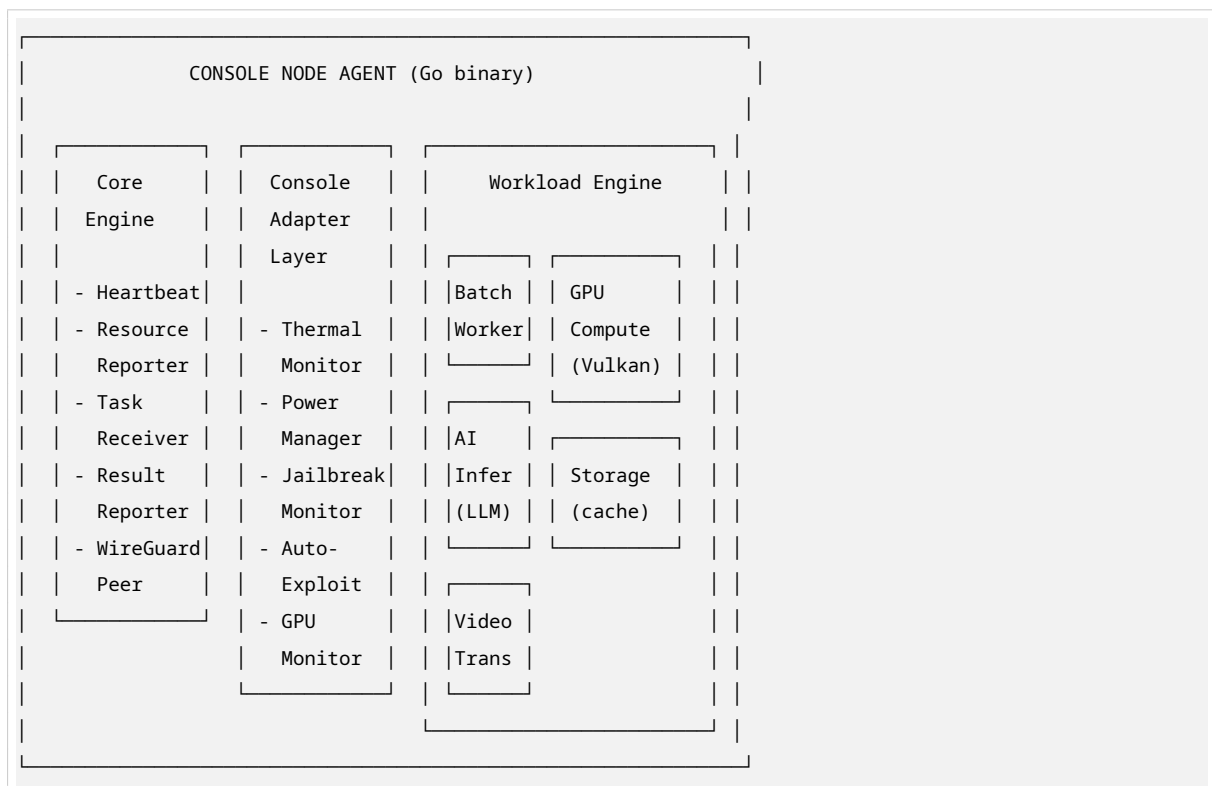




3.2 Console Node Agent Architecture

The Console Node Agent is a Go binary compiled for linux/amd64 that runs as a systemd service on the console's Linux distribution.

Component Diagram



Core Engine

The Core Engine is identical to the PC Node Agent but with console-specific adaptations:


```

package agent

// ConsoleNodeAgent extends the base NodeAgent with console-specific capabilities
type ConsoleNodeAgent struct {
    *BaseNodeAgent // Embedded: heartbeat, resource reporting, task execution

    ConsoleType ConsoleType // PS4_FAT, PS4_PRO, PS5, PS5_PRO
    Adapter      *ConsoleAdapter // Thermal, power, jailbreak management
    TrustLevel   TrustLevel      // Always SEMI for consoles

    // GPU compute (uses same Vulkan backend as PC)
    VulkanBackend *vulkan.ComputeBackend

    // AI inference (llama.cpp subprocess)
    LLMEngine     *llama.InferenceEngine
}

func (a *ConsoleNodeAgent) RegisterWithCluster() error {
    node := &NodeRegistration{
        Type:      NODE_TYPE_CONSOLE,
        ConsoleType: a.ConsoleType,
        TrustLevel: TRUST_SEMI,
        Resources:  a.scanConsoleResources(),
        Capabilities: []Capability{
            {Name: "gpu-vulkan-compute", Type: CAP_GPU},
            {Name: "ai-inference-llama", Type: CAP_AI},
            {Name: "batch-processing", Type: CAP_BATCH},
            {Name: "video-transcode", Type: CAP_VIDEO},
        },
    }
    return a.BaseNodeAgent.Register(node)
}

func (a *ConsoleNodeAgent) scanConsoleResources() *ConsoleNodeResources {
    return &ConsoleNodeResources{
        BaseResources: a.BaseNodeAgent.ScanResources(),
        ConsoleSpecific: ConsoleSpecificInfo{
            Model:      a.ConsoleType,
            Firmware:   a.Adapter.GetFirmwareVersion(),
            JailbreakVersion: a.Adapter.GetJailbreakVersion(),
            Thermal: ThermalState{
                CPUCurrentC: a.Adapter.GetCPUTemp(),
                GPUCurrentC: a.Adapter.GetGPUTemp(),
                FanSpeedPct: a.Adapter.GetFanSpeed(),
                Throttling:   a.Adapter.IsThrottling(),
            },
            Power: PowerState{
                CurrentWatts: a.Adapter.GetPowerConsumption(),
            },
            GPU: GPUState{

```

```

        ClockMHz:      a.Adapter.GetGPUClock(),
        GPUTemperatureC: a.Adapter.GetGPUTemp(),
    },
},
}
}

```

Console Adapter Layer

The Console Adapter is the unique component that handles console-specific hardware:

```

package console

// ConsoleAdapter manages console-specific hardware interfaces
type ConsoleAdapter struct {
    consoleType ConsoleType
    sysfsPath   string        // /sys/class/amdte/ on PS4/PS5
}

// Thermal Management
func (a *ConsoleAdapter) GetCPUTemp() int {
    // Read from /sys/class/thermal/thermal_zone*/temp
    // PS4/PS5 expose thermal zones via standard Linux thermal framework
}

func (a *ConsoleAdapter) GetGPUTemp() int {
    // Read from AMDGPU sysfs: /sys/class/drm/card0/device/hwmon/temp1_input
}

func (a *ConsoleAdapter) SetFanSpeed(percent int) error {
    // Write to /sys/class/hwmon/hwmon*/pwm1
    // Range: 0-100%
}

func (a *ConsoleAdapter) IsThrottling() bool {
    cpuTemp := a.GetCPUTemp()
    gpuTemp := a.GetGPUTemp()
    // PS4 Pro throttles at ~85°C CPU, ~80°C GPU
    // PS5 throttles at ~90°C CPU, ~85°C GPU
    return cpuTemp > 85000 || gpuTemp > 80000 // millidegrees
}

// Power Management
func (a *ConsoleAdapter) GetPowerConsumption() float64 {
    // Read from /sys/class/power_supply/ if available
    // Fallback: estimate from CPU/GPU load + thermal state
}

// Jailbreak Management
func (a *ConsoleAdapter) IsJailbroken() bool {
    // Check if homebrew capabilities are available
}

```

```

    // On Linux: always true (kexec succeeded)
    // On Orbis: check for GoldHen/etaHEN presence
    return a.detectJailbreakMarker()
}

func (a *ConsoleAdapter) TriggerExploit() error {
    // Signal auto-exploit hardware to send payload
    // Via USB serial to ESP32/Luckfox
    // Or: trigger software exploit chain
}

// Auto-exploit via USB serial
func (a *ConsoleAdapter) initAutoExploit() error {
    port, err := serial.Open("/dev/ttyUSB0", &serial.Config{Baud: 115200})
    if err != nil {
        return fmt.Errorf("auto-exploit hardware not found: %w", err)
    }
    // Configure ESP32 for automatic exploit on console boot
    _, err = port.Write([]byte("CONFIG:AUTO_EXPLOIT=ON\n"))
    return err
}

```

3.3 Vulkan Compute Integration

Universal GPU Backend

The most important architectural decision for Phase 2: **no console-specific GPU code is needed**. Our existing Vulkan Compute Backend works on all consoles without modification.

```

package vulkan

// ComputeBackend — same code runs on PC, PS4, PS4 Pro, PS5, PS5 Pro
type ComputeBackend struct {
    instance    vk.Instance
    device      vk.Device
    queue       vk.Queue
    queueFamily uint32
    memoryProps vk.PhysicalDeviceMemoryProperties
}

// Initialize discovers the GPU automatically
func NewComputeBackend() (*ComputeBackend, error) {
    // Vulkan enumerates all devices:
    // On PS4: AMD GCN Liverpool (radv)
    // On PS4 Pro: AMD GCN Polaris (radv)
    // On PS5: AMD RDNA2 Oberon (radv)
    // On PC: Whatever AMD/NVIDIA/Intel GPU is present
    // All use the SAME driver interface
}

// CompileShader compiles GLSL → SPIR-V → GPU binary

```

```
// SPIR-V is the universal intermediate representation
func (b *ComputeBackend) CompileShader(glslSource string) (*Shader, error) {
    // glslangValidator compiles GLSL to SPIR-V
    // SPIR-V is loaded by Vulkan on ANY GPU
}
```

AI Inference: llama.cpp on Console

```
package llama

// InferenceEngine wraps llama.cpp for console AI workloads
type InferenceEngine struct {
    modelPath    string
    gpuLayers    int      // 99 = offload all to GPU
    port         int      // HTTP server port
    process      *os.Process
}

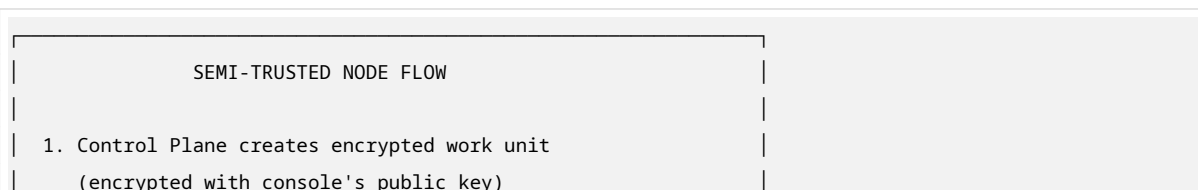
func (e *InferenceEngine) Start() error {
    // Launch llama.cpp server with Vulkan backend
    cmd := exec.Command("/opt/llama.cpp/llama-server",
        "-m", e.modelPath,
        "--gpu-layers", strconv.Itoa(e.gpuLayers),
        "--ctx-size", "8192",
        "--port", strconv.Itoa(e.port),
        "--host", "0.0.0.0",
    )
    // Set Vulkan device selection
    cmd.Env = append(os.Environ(),
        "GGML_VULKAN_DEVICE=0", // Use first Vulkan GPU
    )
    return cmd.Start()
}

// Expected performance:
// PS4:    ~25 tok/s (3B model), ~9 tok/s (7B model)
// PS4 Pro: ~55 tok/s (3B model), ~20 tok/s (7B model)
// PS5:    ~104 tok/s (3B model), ~38 tok/s (7B MoE)
```

3.4 Semi-Trusted Security Model

Architecture

Console nodes operate at `TRUST_LEVEL = SEMI`. This is a deliberate security posture acknowledging that jailbroken consoles have fully compromised kernels.



2. Work unit sent to console via WireGuard
 3. Console decrypts and executes
(runs in sandbox/container)
 4. Console signs result with its key
(ed25519 signature)
 5. Result returned to control plane
 6. Control plane verifies result:
 - a) Cryptographic signature valid?
 - b) LLMsVerifier checks output sanity?
 - c) OR: Redundant compute on trusted node matches?
 7. Only verified results accepted into cluster state
- CONSOLE CANNOT:
- Read cluster state (etcd)
 - Modify any cluster resource
 - Access sensitive data
 - Initiate any cluster operation
 - Communicate with other nodes directly

Implementation

```
package security

// SemiTrustedWorkUnit represents work sent to a console node
type SemiTrustedWorkUnit struct {
    ID          string          `json:"id"`
    Type        WorkType        `json:"type"`      // GPU_COMPUTE, AI_INFERENCE, BATCH
    EncryptedPayload []byte        `json:"payload"`   // Encrypted with console pubkey
    Environment map[string]string `json:"env"`       // Container environment
    Timeout     time.Duration   `json:"timeout"`
    VerifyMode  VerifyMode      `json:"verify_mode"` // LLM_VERIFY or REDUNDANT
}

type SemiTrustedResult struct {
    WorkUnitID string          `json:"work_unit_id"`
    Output      []byte         `json:"output"`
    Signature   []byte         `json:"sig"`        // ed25519 signature
    Metrics     WorkMetrics     `json:"metrics"`    // Duration, GPU util, etc.
    ConsoleID   string          `json:"console_id"`
    Timestamp   time.Time       `json:"timestamp"`
}

func (s *SecurityManager) VerifyConsoleResult(result *SemiTrustedResult) error {
    // 1. Verify signature
```

```

    if !ed25519.Verify(consolePubkey, result.Output, result.Signature) {
        return ErrInvalidSignature
    }

    // 2. Check timestamp freshness (prevent replay)
    if time.Since(result.Timestamp) > 5*time.Minute {
        return ErrResultStale
    }

    // 3. Mode-specific verification
    switch result.VerifyMode {
    case LLM_VERIFY:
        // Use LLMsVerifier to check output sanity
        return s.llmVerifier.CheckOutput(result.Output)
    case REDUNDANT:
        // Compare with trusted node's result
        return s.compareWithTrusted(result)
    case TRIVIAL:
        // No verification needed (already trivial/known result)
        return nil
    }

    return nil
}

```

3.5 Scheduler Integration: Console-Aware Plugin

The scheduler needs to be aware of console-specific constraints:

```

package scheduler

// ConsoleAwarePlugin prevents scheduling inappropriate workloads on consoles
type ConsoleAwarePlugin struct {
    thermalThreshold int // Celsius
}

func (p *ConsoleAwarePlugin) Filter(ctx context.Context,
    state *framework.CycleState, pod *v1.Pod,
    nodeInfo *framework.NodeInfo) *framework.Status {

    node := nodeInfo.Node()

    // Check if target is a console node
    if node.Labels["node-type"] != "console" {
        return framework.NewStatus(framework.Success) // Not a console, allow
    }

    // Console-specific filters:

    // 1. Don't schedule AVX2-required workloads on PS4
    if node.Labels["console-model"] == "ps4" || node.Labels["console-model"] == "ps4-pro" {

```

```

        if requiresAVX2(pod) {
            return framework.NewStatus(framework.Unschedulable,
                "PS4 lacks AVX2 support")
        }
    }

    // 2. Don't schedule >8GB RAM workloads on PS4
    if node.Labels["console-tier"] == "3" && memoryRequest(pod) > 6*GiB {
        return framework.NewStatus(framework.Unschedulable,
            "PS4 has limited RAM")
    }

    // 3. Don't schedule sensitive data workloads on any console
    if containsSensitiveData(pod) {
        return framework.NewStatus(framework.Unschedulable,
            "Console nodes cannot access sensitive data")
    }

    // 4. Check thermal state
    if isConsoleOverheating(node) {
        return framework.NewStatus(framework.Unschedulable,
            "Console thermal throttling")
    }

    // 5. Console nodes only get idempotent workloads
    if !isIdempotent(pod) {
        return framework.NewStatus(framework.Unschedulable,
            "Console nodes require idempotent workloads")
    }

    return framework.NewStatus(framework.Success)
}

func (p *ConsoleAwarePlugin) Score(ctx context.Context,
    state *framework.CycleState, pod *v1.Pod,
    nodeInfo *framework.NodeInfo) (int64, *framework.Status) {

    node := nodeInfo.Node()
    if node.Labels["node-type"] != "console" {
        return 0, nil // No score modification for non-consoles
    }

    score := int64(100)

    // Penalize overheating consoles
    cpuTemp, _ := getCPUTemp(node)
    if cpuTemp > 80 {
        score -= int64(cpuTemp - 80) * 5 // -5 points per degree over 80
    }

    // Bonus for consoles with good thermal headroom

```

```

    if cpuTemp < 60 {
        score += 20
    }

    // Prefer PS5 for GPU-intensive workloads
    if isGPUWorkload(pod) && node.Labels["console-tier"] == "1" {
        score += 50
    }

    return score, nil
}

```

3.6 Health Monitoring: Console-Specific Metrics

```

# Console-specific Prometheus metrics
# These are collected by the Console Adapter and exposed by the node agent

# Temperature metrics
console_cpu_temperature_celsius{node_id="ps4-pro-001"} 72
console_gpu_temperature_celsius{node_id="ps4-pro-001"} 68
console_fan_speed_percent{node_id="ps4-pro-001"} 45

# Power metrics
console_power_consumption_watts{node_id="ps4-pro-001"} 142.5
console_power_daily_kwh{node_id="ps4-pro-001"} 2.8

# GPU metrics (console-specific)
console_gpu_clock_mhz{node_id="ps4-pro-001"} 911
console_gpu_vram_used_bytes{node_id="ps4-pro-001"} 2147483648
console_gpu_throttling{node_id="ps4-pro-001"} 0

# Jailbreak metrics
console_jailbreak_active{node_id="ps4-pro-001"} 1
console_jailbreak_version{node_id="ps4-pro-001", version="2.4b"} 1
console_linux_uptime_seconds{node_id="ps4-pro-001"} 172800

# Storage health
console_ssd_health_percent{node_id="ps4-pro-001"} 94
console_ssd_wear_level{node_id="ps4-pro-001"} 6
console_ssd_power_on_hours{node_id="ps4-pro-001"} 8760

# Thermal throttling alerts
- alert: ConsoleThermalThrottling
  expr: console_cpu_temperature_celsius > 85 or console_gpu_temperature_celsius > 80
  for: 2m
  labels:
    severity: warning
  annotations:
    summary: "Console {{ $labels.node_id }} is thermal throttling"

- alert: ConsoleJailbreakLost

```



```
expr: console_jailbreak_active == 0
for: 1m
labels:
  severity: critical
annotations:
  summary: "Console {{ $labels.node_id }} lost jailbreak"
```

3.7 Auto-Exploit Hardware Integration

For unattended console cluster nodes, auto-exploit hardware automates the jailbreak process:

ESP32 Auto-Exploit Setup

```
// ESP32 firmware for automatic PS4/PS5 jailbreak
// Connects to console USB port, sends exploit on boot detection

#include <USB.h>
#include <PS4Exploit.h> // Custom exploit payloads

const int CONSOLE_POWER_SENSE_PIN = 4; // GPIO connected to console power LED
bool consolePowered = false;

void setup() {
  pinMode(CONSOLE_POWER_SENSE_PIN, INPUT);
  Serial.begin(115200);

  // Load exploit payload for target firmware
  loadPayload("goldhen_2.4b_900.bin");
}

void loop() {
  bool currentPower = digitalRead(CONSOLE_POWER_SENSE_PIN);

  if (currentPower && !consolePowered) {
    // Console just powered on — send exploit
    Serial.println("Console boot detected, sending exploit...");
    delay(5000); // Wait for USB stack initialization
    sendExploitPayload();
    consolePowered = true;
  }

  if (!currentPower && consolePowered) {
    // Console powered off
    consolePowered = false;
  }

  delay(1000);
}
```

Provisioning Integration

The setup wizard for console nodes includes auto-exploit hardware configuration:

```
$ htmux cluster add-console --auto-exploit

[Auto-Exploit Setup]
1. Connect ESP32 to console's front USB port
2. Connect ESP32 to cluster management network (WiFi)
3. Flashing auto-exploit firmware...
   [██████████] 100%
4. Configuring for firmware 9.00 (detected)...
5. Testing auto-exploit cycle...
   Power off → Power on → Exploit sent $${\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{\usr/
   ↳ share/docminer/emoji/2713.png}}}}$ → GoldHen loaded
   ↳ ${\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{\usr/share/docminer/emoji/
   ↳ 2713.png}}}}$$
6. Console will now auto-jailbreak on every boot.
```

Chapter 4: Phase 2 Implementation Plan & Console Setup

4.1 Phase 2 Task Breakdown

HelixCluster Phase 2 adds **24 new tasks** (~176 hours, ~4.5 weeks) to the existing Phase 0-8 plan. These tasks are distributed across all phases, with the heaviest concentration in Phase 0 (foundations) and Phase 1 (infrastructure).

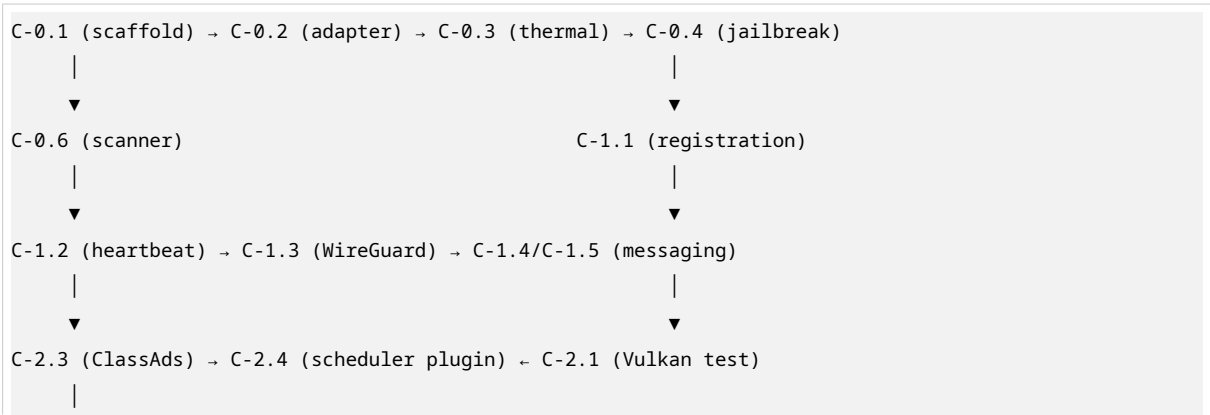
Console-Specific Task Matrix

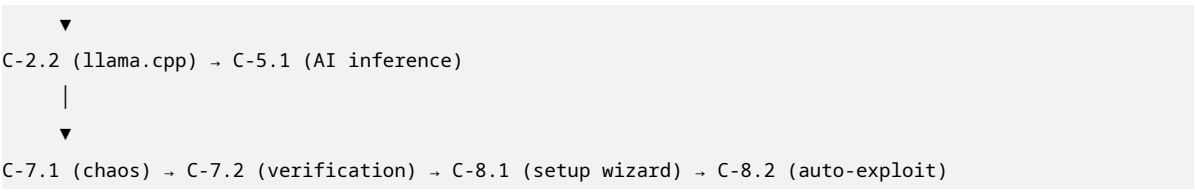
Phase	Task ID	Description	Hours	Priority	Skill
0	C-0.1	Console Agent Go project scaffolding	8h	P0	GO
0	C-0.2	ConsoleAdapter interface definition	4h	P0	GO
0	C-0.3	Thermal/power monitoring via sysfs/hwmon	8h	P0	GO
0	C-0.4	Jailbreak detection library	8h	P0	GO
0	C-0.5	Auto-exploit ESP32 firmware (C++)	16h	P1	C
0	C-0.6	Console capability scanner	4h	P0	GO
0	C-0.7	PS5 Orbis I/O Agent (native)	16h	P2	C
1	C-1.1	Console node registration (SEMI trust)	4h	P0	GO
1	C-1.2	Console heartbeat with thermal metrics	4h	P0	GO

Table 6 – continued

Phase	Task ID	Description	Hours	Priority	Skill
1	C-1.3	WireGuard kernel module for PS4/PS5 Linux	4h	P0	GO
1	C-1.4	ZeroMQ lightweight client for PS4	4h	P0	GO
1	C-1.5	gRPC client for PS5	4h	P0	GO
2	C-2.1	Vulkan Compute Backend validation on PS4/PS5	8h	P0	C
2	C-2.2	llama.cpp Vulkan integration for consoles	8h	P0	C
2	C-2.3	Console-specific ClassAds expressions	4h	P0	GO
2	C-2.4	ConsoleAware scheduler plugin	8h	P0	GO
3	C-3.1	Minimal PTY session backend for consoles	8h	P1	GO
4	C-4.1	AOSP distcc worker on PS4	8h	P1	GO
4	C-4.2	AOSP distcc + GPU worker on PS5	8h	P1	GO
5	C-5.1	AI inference agent (llama.cpp server)	8h	P1	GO
7	C-7.1	Console chaos tests (power loss, thermal)	8h	P0	QA
7	C-7.2	SEMI trust model verification testing	8h	P0	QA
8	C-8.1	Console setup wizard (htmux add-console)	8h	P0	GO
8	C-8.2	Auto-exploit hardware provisioning	8h	P1	GO

Critical Path for Phase 2





Integration Points with Existing Components

Existing Component	Console Integration	Effort
Node Discovery	Add CONSOLE node type, SEMI trust level	Low
Resource Scheduler	ConsoleAware plugin (thermal, AVX2, RAM filters)	Medium
GPU Compute Engine	No changes –Vulkan backend is universal	None
Health Monitor	Add console-specific metrics (thermal, power, SSD)	Medium
LLM Brain	Console AI inference pool for parallel agents	Low
Build Service	Console distcc workers for AOSP	Low
Security Manager	SEMI trust model, encrypted work units	Medium
Session Manager	Minimal PTY for console nodes (no migration)	Low

4.2 Console Setup Wizard

The console setup wizard is invoked via `htmux cluster add-console` and automates the entire provisioning process.

Phase 1: Discovery

```
$ htmux cluster add-console --discover

Scanning local network for PlayStation consoles...

[DISCOVERED CONSOLES]
+---+-----+-----+-----+-----+-----+
| ## | Model  | IP Address | MAC Address | Firmware | Status |
+---+-----+-----+-----+-----+-----+
| 01 | PS4 Pro | 192.168.1.45 | A4:17:31:XX:XX:XX | 9.00 | JB |
| 02 | PS4 Fat | 192.168.1.47 | A4:17:31:XX:XX:XX | 11.00 | No JB |
| 03 | PS5     | 192.168.1.50 | 88:C9:E8:XX:XX:XX | 4.51 | JB |
+---+-----+-----+-----+-----+-----+

Note: PS4 at 192.168.1.47 (firmware 11.00) cannot be jailbroken.
      Only firmwares ≤9.00 (PS4) or ≤4.51 (PS5) are exploitable.

Select consoles to add (comma-separated): 1,3
```

Phase 2: Jailbreak

```
[PS4 Pro at 192.168.1.45]
Firmware: 9.00 (GoldHen-compatible)
Preparing jailbreak payload...
```

```
[██████████████████] 100%
GoldHen v2.4b loaded successfully
Debug settings enabled
FTP server active on port 2121
BinLoader active on port 9090
```

```
[PS5 at 192.168.1.50]
Firmware: 4.51 (etaHEN-compatible)
Preparing jailbreak payload...
```

```
[██████████████████] 100%
etaHEN loaded successfully
Homebrew enabled
FTP server active on port 2121
```

Phase 3: Linux Installation

```
[Installing Linux on PS4 Pro]
Downloading psxitar v3 (kernel 6.15.4)...
[██████████████████] 100% 1.2 GB downloaded
```

```
Preparing USB drive /dev/sdb...
Writing Linux payload...
[██████████████████] 100%
```

```
Booting Linux via kexec...
[██████████████████] 100%
Linux 6.15.4-ps4 booted successfully
8 CPU cores detected
AMDGPU loaded (36 CUs)
6.85 GB RAM available
Gigabit Ethernet: UP (1000 Mbps)
```

```
[Installing Linux on PS5]
Downloading Ubuntu 24.04 for PS5...
[██████████████████] 100% 2.1 GB downloaded
```

```
Writing to USB drive...
Booting via ps5-linux-loader...
[██████████████████] 100%
Ubuntu 24.04 booted successfully
16 CPU threads detected (Zen 2)
AMDGPU loaded (RDNA2, 36 CUs)
12.5 GB RAM available
Gigabit Ethernet: UP (1000 Mbps)
```

M.2 SSD: detected

Phase 4: Agent Installation

```
[Installing HelixCluster Console Agent]

Downloading console-agent-linux-amd64...
[████████████████████] 100%

Installing systemd service...
  Creating user: helix (no sudo)
  Installing binary: /opt/helix/bin/console-agent
  Creating service: /etc/systemd/system/helix-console.service
  Enabling auto-start:

Configuring agent...
  Control plane: auto-discovered at 192.168.1.10:8443
  WireGuard mesh: generating keys...
  Node labels: tier=2,model=ps4-pro

Starting agent...
[████████████████████] 100%
Agent running, PID 1847
```

Phase 5: Cluster Registration

```
[Registering with HelixCluster]

PS4 Pro (192.168.1.45):
  Node ID: c4f8e2d1-7a3b-4c5d-9e0f-1a2b3c4d5e6f
  Trust Level: SEMI
  Tier: 2 (Standard)
  WireGuard IP: 100.64.2.15

  ┌────────────────────────────────────────────────────────────────────────────────┐
  │ CAPABILITIES                                                                    │
  │ gpu-vulkan-compute (GCN 4.0, 4.2TF) │                                         │
  │ ai-inference-llama (55 tok/s 3B)   │                                         │
  │ batch-processing (8x Jaguar)        │                                         │
  │ video-transcode (GPU shader)        │                                         │
  └────────────────────────────────────────────────────────────────────────────────┘

PS5 (192.168.1.50):
  Node ID: d5e9f3g2-8b4c-5d6e-0a1b-2c3d4e5f6a7b
  Trust Level: SEMI
  Tier: 1 (Premium)
  WireGuard IP: 100.64.2.16

  ┌────────────────────────────────────────────────────────────────────────────────┐
  │ CAPABILITIES                                                                    │
  │ gpu-vulkan-compute (RDNA2, 10.3TF) │                                         │
  │ ai-inference-llama (104 tok/s 3B)  │                                         │
  │ batch-processing (Zen2 8c/16t)     │                                         │
  └────────────────────────────────────────────────────────────────────────────────┘
```

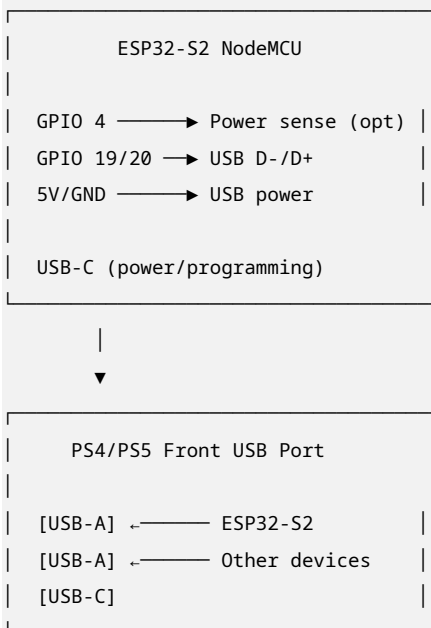
```
| video-transcode (GPU shader) |  
| hardware-decompress (Kraken, Orbis) |  
└───────────────────────────────────┘
```

2 console nodes successfully added to HelixCluster!

Cluster GPU TFLOPS: +14.5 (4.2 from PS4 Pro, 10.3 from PS5)
Cluster CPU cores: +24 (8 from PS4 Pro, 16 from PS5)
Cluster RAM: +20.35 GB

View status: [htmux cluster status](#)

Component	Model	Cost	Purpose
ESP32-S2 DevKit	NodeMCU-32-S2	\$4.50	Auto-exploit MCU
USB-A Cable	1ft right-angle	\$2.00	Connect to console
3D Printed Case	Custom design	\$1.00	Enclosure
JST Connector	2-pin	\$0.50	Power sense wire
Total per kit		~\$8	



```
$ htmux cluster provision-auto-exploit --device /dev/ttyUSB0 --console ps4-pro-001

Flashing auto-exploit firmware to ESP32...
Chip: ESP32-S2 (revision 0)
Flash size: 4MB
[████████████████████] 100%

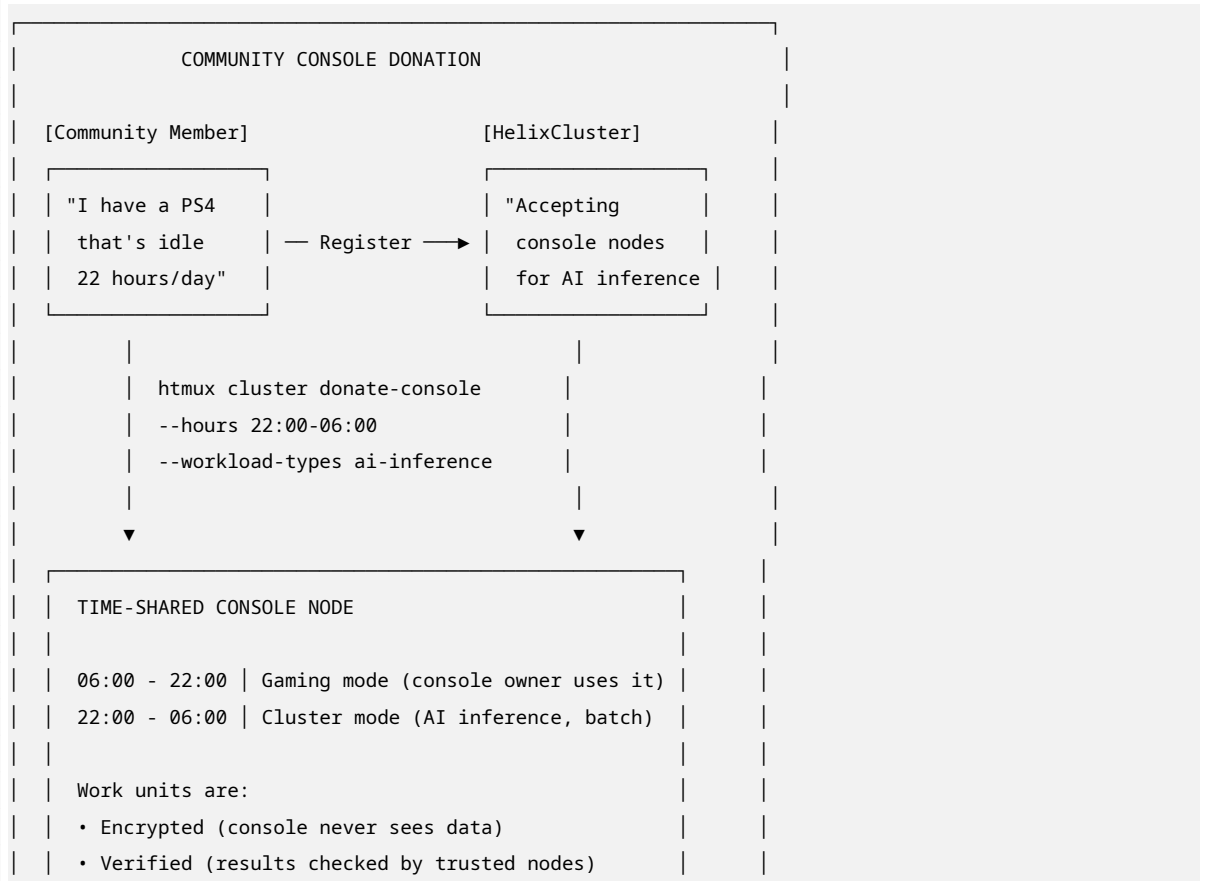
Configuring:
Target firmware: 9.00 (from console registration)
Exploit type: GoldHen (USB method)
Auto-trigger: ON (power sense)
LED indicator: ON

Testing:
Simulating console boot...
Exploit payload sent $$\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/
↪ emoji/2713.png}}}}}$
Expected GoldHen load: ~8 seconds

$\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/2713.png}}}}$$
↪ Auto-exploit hardware provisioned for ps4-pro-001
```

4.4 Community Console Donation Model

A unique capability enabled by the semi-trusted model: **community members can donate idle console time.**



	• Interruptible (gaming takes priority)		
	• Compensated (owner receives compute credits)		

4.5 Performance Validation Targets

Acceptance Criteria for Phase 2

Test	Target	Measurement
PS4 Pro Vulkan compute	3.5 TFLOPS sustained	clpeak benchmark
PS5 Vulkan compute	8.5 TFLOPS sustained	clpeak benchmark
llama.cpp PS4 Pro (3B)	50 tok/s	llama-bench
llama.cpp PS5 (3B)	100 tok/s	llama-bench
AOSP build contribution	8 parallel jobs	distcc monitor
Network throughput	850 Mbps	iperf3
Thermal stability (24h)	<80°C CPU sustained	stress-ng + monitoring
Jailbreak persistence	30+ days in REST mode	Longevity test
Auto-exploit reliability	99%+ success rate	100 boot cycles
Console agent memory	<128 MB RAM	ps/aux measurement
Workload verification	100% accuracy	Redundant compute check
Cluster integration	Same APIs as PC nodes	End-to-end test